

Débogage de la chaîne d'acquisition

COLBERTo — SilvaScience, Université de Montréal
Caméras Stresing et Pixis, spectrographe SpectraPro 2300i
Session du 1^{er} août 2026 — branche Bug_Streising

1. Résumé

La session a commencé sur un symptôme unique — la caméra Stresing refusait de s'initialiser avec l'erreur *Getting DMA buffer failed* — et a mis au jour une vingtaine de défauts indépendants, regroupés en treize correctifs. Plusieurs affectaient **tous les spectromètres** depuis longtemps sans avoir jamais été identifiés : le plus grave empêchait purement et simplement l'affichage du moindre spectre, un autre corrompait silencieusement les données corrigées du fond.

Deux des correctifs ont eux-mêmes introduit des régressions, corrigées à leur tour et documentées en section 3.9. Elles méritent d'être lues : elles reproduisent exactement le défaut que ce rapport décrit, ce qui en dit long sur la facilité avec laquelle il se glisse dans ce code.

Le fil conducteur de ces bugs est le même : **des échecs silencieux**. Une exception envoyée sur la sortie d'erreur plutôt que dans le journal, une commande série invalide à laquelle l'instrument répond ok, un appel bloquant sans échéance. Dans chaque cas le logiciel affichait un état normal alors que rien ne fonctionnait.

État final

La Stresing acquiert et affiche un spectre en mode continu ; le pic d'un laser hélium-néon à 632,8 nm a été observé sur les deux caméras, ce qui valide la chaîne complète, de la carte PCIe jusqu'au graphique.

2. Le fil du diagnostic

Le symptôme s'est déplacé cinq fois. Chaque correctif révélait le défaut suivant, jusque-là masqué. Cette chronologie explique pourquoi le problème avait résisté aux sessions précédentes.

Étape	Observation	Cause réelle
1	Stresing refuse de démarrer : <i>Getting DMA buffer failed</i>	Taille du buffer DMA jamais assignée (corrigé la veille)
2	Le logiciel s'ouvre, l'interface gèle complètement	Lecture matérielle bloquante exécutée dans le thread graphique
3	Interface réactive, mais progression figée à 50 %	La carte attendait un trigger externe absent, sans échéance
4	Acquisition terminée en moins d'une seconde, aucun spectre affiché	IndexError silencieuse avant l'émission du signal
5	Spectre affiché sur la Stresing, plat sur la Pixis	Miroir de sortie du spectrographe dirigé vers l'autre port

L'étape 4 mérite un commentaire. Le journal affichait `Begin single spectrum acquisition` puis `Finished` : tout semblait parfait. L'exception se produisait dans un connecteur Qt, dont la trace part sur la sortie d'erreur standard et non dans `main.log`. Le journal, seule source consultée jusque-là, était structurellement incapable de montrer le problème.

3. Correctifs apportés

Treize commits sur la branche Bug_Streising, séparés par sujet.

3.1 Les spectres n'atteignaient jamais le graphique

e6be751 — DataHandling.py, SpectrometerPlot.py

Le défaut le plus grave de la session, et le plus ancien. `concatenate_data()` lisait `queue[-1]` sur chaque file de paramètres sans vérifier qu'elle contenait une valeur. Or `update_parameter()` n'est appelée **null** part dans le code : ces files sont donc toujours vides, et la lecture levait une `IndexError` à chaque spectre, avant l'émission du signal vers le graphique.

Conséquence : **aucun spectre ne s'est jamais affiché**, quel que soit le spectromètre, tant que ce code existait. La mesure se terminait normalement, le journal écrivait `Finished`, et rien n'apparaissait.

Second défaut du même commit : le graphique se peuplait au démarrage d'une gaussienne aléatoire s'étalant de 177 à 884 nm. Cette courbe restait en place et imposait cette plage aux axes ; un spectre Stresing réel, large de 48 nm, était comprimé dans une bande illisible.

3.2 Gel de l'interface

bb62b91 — main.py

Lorsqu'une mesure était déjà en cours, `acquire_measurement()` appelait `take_spectrum()` directement. Cette fonction exécute la lecture matérielle, et ce chemin s'exécute dans le thread graphique : l'interface restait donc figée aussi longtemps que la caméra mettait à répondre — c'est-à-dire indéfiniment lorsque la carte attendait un trigger absent.

Le diagnostic a été obtenu avec `py-spy`, qui donne la pile d'appels Python d'un processus vivant :

```
Thread 19404 (active)
  measure (StresingDriver.py:195)
  acquire_spectrum (Stresing.py:349)
  get_intensities (Stresing.py:333)
  run (MeasurementClasses.py:66)
```

3.3 Stresing : modes de trigger, échéance, worker inopérant

df13fc9 — Stresing.py, StresingDriver.py

Worker court-circuité. `get_intensities()` appelait `StresingWorker.worker_spectrum` en méthode non liée, en lui passant la caméra comme `self`. Le résultat était donc écrit sur l'objet caméra pendant que la boucle du worker attendait un drapeau que personne ne levait : le signal `sendSpectrum` n'était jamais émis. La boucle d'attente qui suivait aurait tourné à l'infini si elle avait été atteinte, puisqu'elle remettait à zéro le drapeau qu'elle surveillait.

Échéance d'acquisition. L'appel bloquant du DLL ne peut pas être interrompu. Il n'est désormais utilisé que lorsque la carte se déclenche elle-même et que les lectures sont garanties ; toute attente d'un signal externe passe par un sondage avec échéance, qui abandonne en indiquant combien de lectures sont arrivées :

```
TimeoutError: No scan received after 5.0 s (scan 0 of 10, block 1 of 1).
The board is waiting for a trigger that is not arriving.
```

Modes de trigger. Les registres `sti` et `bti` partagent la numérotation des entrées externes mais divergent au-delà de 4, et chacun possède un temporisateur qui ne s'applique que dans un seul de ses modes. Exposés en huit champs numériques indépendants, ils étaient inutilisables : rien n'indiquait que `Scan_Timer` est ignoré tant que `Scan_Trig` ne vaut pas exactement 4. Le réglage se fait désormais par mode.

3.4 SpectraPro : le miroir de sortie ne bougeait pas

fd4eb09 — SpectraPro2300i.py

Le spectrographe possède un miroir motorisé qui choisit par quel port de sortie la lumière ressort. La Stresing est sur le port *side*, la Pixis sur le port *front* : la caméra située sur l'autre port ne reçoit rien. Le pilote envoyait la commande <n> MIRROR, que le contrôleur accepte sans rien faire.

Vérification directe sur l'instrument (SP-2-300i, série 23581080), en affichant les réponses brutes plutôt que filtrées :

```
FRONT -> ' ok'
?MIRROR -> ' front' le miroir a bouge
1 MIRROR -> ' ok'
?MIRROR -> ' front' il n'a PAS bouge
```

Les commandes correctes sont EXIT-MIRROR puis FRONT ou SIDE. Ce défaut était masqué par la fonction de dialogue série, qui ne retourne que les chiffres de la réponse : un message d'erreur, ou une réponse textuelle comme *front*, est invisible pour le logiciel.

Deux autres correctifs dans le même fichier. Le nombre de réseaux était déduit de la longueur de la réponse et en comptait 4 sur une tourelle qui en porte 3, inventant un réseau à 4 traits/mm ; MONO-EESTATUS confirme 1200, 1200 et 300 traits/mm. Et la boucle de lecture série attendait le caractère de fin sans aucune porte de sortie : un contrôleur muet figeait le thread graphique indéfiniment.

3.5 Panneau d'acquisition et réorganisation de l'onglet

319c428 — AcquisitionSettings.py, main.py

Nouveau panneau remplaçant les champs de registres bruts. Il propose les trois configurations réellement utilisées — carte sur son temporisateur interne, une lecture par impulsion laser, blocs cadencés par un hacheur — et en déduit les registres. Les intervalles se saisissent avec leur unité et affichent la fréquence et la durée totale ; les réglages qu'un mode n'utilise pas sont masqués plutôt que laissés à l'écran sans effet ; une ligne dépliant montre exactement ce qui sera écrit dans la carte. Le panneau porte aussi le monochromateur : longueur d'onde centrale, réseau, et port de sortie nommé d'après la caméra qu'il alimente.

L'onglet *Spectrum View* est composé en code, sans toucher au fichier Qt Designer que plusieurs personnes éditent et qui fusionne mal. Un second onglet ne contient que le graphique, pour la lecture à distance en laboratoire.

3.6 La soustraction de fond corrompait les spectres

23f7ad7 — DataHandling.py, main.py, MeasurementClasses.py

Ce défaut produisait des chiffres faux plutôt qu'une panne visible, ce qui en fait le plus dangereux de la session. Le tableau de fond était alloué avec `np.empty`, qui ne remplit rien et laisse le contenu antérieur de la mémoire, et **aucune mesure de fond ne le remplissait** : la mesure envoyait son résultat vers l'enregistrement des données ordinaires, la version de calibration vers le dictionnaire de calibration. Seul le chargement d'un fichier assignait réellement ce tableau.

Acquérir un fond dans la session puis cocher la correction soustrayait donc des valeurs arbitraires de chaque spectre, sans erreur et sans trace dans le journal. Lors d'un essai de vérification, ce tableau avait une moyenne de 99,8 comptes.

Le changement de spectromètre aggravait le tout en réallouant le même tableau non initialisé, pendant que la case de correction restait cochée : un fond correctement chargé redevenait silencieusement de la mémoire brute.

Le principe retenu est de **ne jamais soustraire ce dont on n'est pas sûr**. Le tableau est désormais mis à zéro et protégé par un indicateur ; la correction est ignorée, avec un avertissement, tant qu'aucun fond n'a été acquis ou chargé, ou si sa longueur ne correspond pas. La mesure de fond enregistre réellement son résultat, un fichier de mauvaise longueur est refusé plutôt qu'accepté en silence, et changer de spectromètre abandonne le fond en le disant. Un spectre non corrigé se voit et se rattrape ; un spectre corrigé avec de la mémoire aléatoire, non.

La mesure de fond elle-même ne lisait les longueurs d'onde qu'à l'intérieur de sa boucle de moyennage : un fond d'un seul balayage émettait un tableau vide. Son champ de saisie ne déclarant aucun minimum, zéro balayage était accepté et divisait par zéro.

3.7 Paramètres non enregistrés, interface bloquée, sauvegarde en course

8018894 — DataHandling.py, main.py, MeasurementClasses.py

Paramètres matériels. `update_parameter()` attendait une séquence indexée par position et n'était appelée de nulle part. Ses files restaient donc vides et la matrice de paramètres ne contenait que des zéros : **aucun réglage n'était enregistré à côté des données**, rendant impossible de retrouver après coup les conditions d'une mesure. Elle accepte maintenant un dictionnaire et est appelée toutes les demi-secondes, en fusionnant les valeurs en lecture seule de l'Updater avec les réglages détenus par la fenêtre principale. Les clés absentes et les lectures non numériques — le cryostat qui répond *Stable* — répètent la dernière valeur, pour que les colonnes restent alignées sur les spectres.

Blocage de l'interface. `measurement_busy` n'est libéré que sur une progression à 100 %, que chaque mesure n'émettait qu'après une exécution réussie. Une exception ou un arrêt anticipé laissait donc l'interface répondre *devices are busy* jusqu'au redémarrage du logiciel, et la trace partait sur la sortie d'erreur plutôt que dans le journal. Les quatre classes de mesure journalisent désormais leurs exceptions et émettent la progression finale depuis un bloc `finally`.

Sauvegarde. `save_data()` confiait l'écriture à un autre thread, dormait une demi-seconde, puis copiait le fichier. Sur un gros tampon ou un disque lent, la copie capturait un fichier à moitié écrit. Elle attend maintenant un événement posé par l'écrivain et refuse de copier si celui-ci n'arrive pas en 60 s. En pratique elle rend la main en une vingtaine de millisecondes au lieu de geler l'interface une demi-seconde à chaque sauvegarde.

3.8 Pixis : la caméra pilotée par deux threads à la fois

a3042ba — Pixis.py, SpectrometerPlot.py

PICam refuse de démarrer une acquisition déjà en cours. Or `get_intensities()` démarrait et arrêta la caméra à chaque appel depuis le thread de mesure, pendant que `set_roi()` faisait la même chose depuis le thread graphique quand une zone de lecture était appliquée. Appliquer une région pendant une vue en direct faisait donc échouer la mesure avec *AcquisitionInProgress*. Les deux passent maintenant par un verrou réentrant, et démarrer ou arrêter interroge l'état réel de la caméra plutôt qu'un drapeau. Trois threads qui démarrent et arrêtent pendant que soixante changements de région sont appliqués ne lèvent plus rien, là où ce schéma reproduisait l'erreur.

L'attente d'une image était par ailleurs sans limite, et le message d'échec du worker — *Spectrum not sent from Worker* — ne nommait ni la cause ni l'état de la caméra.

3.9 Deux régressions introduites par ces correctifs

7e0bb5f, 6c863b1 — Stresing.py, StresingDriver.py

Le délai d'attente ajouté en 3.3 appelait `DLLStopMeasurement` pour arrêter la carte. **Cette fonction n'existe pas dans ce DLL**, et l'appel était entouré d'un `try/except AttributeError` qui avalait l'erreur. Chaque expiration abandonnait donc l'attente sans jamais arrêter la carte : la mesure suivante échouait avec *Measurement is already running*, ou bloquait sans retour. Le symptôme observé au laboratoire — « la

Stresing n'affiche plus rien » — venait de là. L'export réel est DLLAbortMeasurement, vérifié sur l'instrument.

La correction de cette première régression en a produit une seconde : pour supprimer tout risque de blocage, l'appel bloquant a été retiré au profit du chemin non bloquant. Or DLLStartMeasurement_nonblocking renvoie **976, « An unknown error occurred »** sur cette carte, et son code de retour n'était vérifié nulle part. Aucune mesure ne démarrait plus. L'appel bloquant, lui, rend la main en 0,11 s avec un spectre valide.

La solution retenue garde l'appel bloquant, seul fonctionnel, mais ne s'y engage jamais à l'aveugle. Sur timer interne les lectures sont garanties. Sur trigger externe, la carte est d'abord interrogée pour savoir si un trigger est réellement apparu, via DLLResetScanTriggerDetected et DLLGetScanTriggerDetected : une entrée muette est signalée au lieu de figer l'appelant. Ces deux fonctions prennent un pointeur vers le drapeau en second argument ; les appeler avec le seul numéro de carte s'exécute et corrompt la mémoire, ce qu'une violation d'accès a permis de découvrir.

Ces deux régressions sont instructives. Elles ont été commises en corrigeant précisément ce type de défaut, avec le rapport sous les yeux, et elles en reprennent la forme exacte : une erreur avalée par un except trop large, puis un code de retour ignoré. Le code appelle un DLL dont les échecs ne se voient pas si on ne les regarde pas.

3.10 Les réglages caméra rejoignent l'onglet d'acquisition

fccfec7 — main.py, AcquisitionSettings.py, Pixis.py

Les paramètres propres à la caméra s'éditent désormais dans le panneau d'acquisition, à côté des réglages qu'ils affectent, plutôt que dans un arbre partagé avec tous les autres appareils. Les contrôles sont construits à partir du `parameter_display_dict` du pilote, donc sans code propre à chaque caméra : la Pixis expose son temps de pose et son nombre de moyennages, la Stresing son gain ADC et ses convertisseurs. Les paramètres que le panneau présente déjà sous une forme plus lisible ne sont pas répétés.

Les lignes du spectromètre dans l'arbre matériel rejoignent le cryostat en lecture seule, tout en restant rafraîchies. Le SLM et les autres appareils sont inchangés. Un changement fait dans le panneau est répercuté vers la valeur enregistrée avec les données, sans quoi le fichier conserverait un réglage périmé.

Défaut trouvé au passage : la température du capteur Pixis était déclarée modifiable alors que rien ne la règle et que le worker seul la met à jour. Elle se retrouvait donc parmi les paramètres que l'actualiseur n'interroge pas, et la valeur affichée ne changeait jamais après le démarrage.

3.11 Le spectre binné qui semblait tomber à zéro

3c7adf9 — Pixis.py, CameraDisplay.py, main.py

Un spectre binné paraissait plat à zéro après un spectre plein cadre. Il ne l'était pas. Mesuré sur le banc : le plein cadre donne 200 896 à 16 513 997 comptes, le binné 1636 à 65 535. Le plein cadre somme ses 252 lignes en Python, sans rien pour l'écrêter ; le binné somme sur la puce et bute sur la limite du convertisseur. Les deux diffèrent donc d'un facteur voisin de 250, et comme le graphe accumule les courbes, celle du plein cadre s'appropriait les axes et la binnée était dessinée le long de l'axe horizontal.

Appliquer une zone de lecture vide désormais les deux graphes, et l'événement est émis pour le plein cadre comme pour le binné, ce qui n'était pas le cas. La ligne d'état indique dans chaque mode d'où viennent les comptes et que les deux ne se comparent pas.

Le temps de pose était converti par `int()` avant d'atteindre la caméra : toute valeur sous la milliseconde devenait zéro. Relu depuis l'instrument, 0,5 ms arrivait comme 0,0 alors que 1, 5, 20 et 100 ms passaient correctement. C'est précisément la plage utile sur ce montage.

4. Ce qui reste ouvert

Les défauts ci-dessous ont été identifiés mais délibérément laissés en l'état, faute d'une information que seul l'utilisateur du montage peut fournir, ou parce qu'ils dépassent le cadre de cette session.

Sujet	État	Ce qu'il faut décider
Fond des scans de chirp	AcquireBackground envoie toujours son résultat au seul dictionnaire de calibration.	Ce fond est-il destiné à la soustraction générale ou uniquement à la calibration ? Les deux usages sont légitimes et le code ne tranche pas.
Calibration de la Pixis	Le fichier d'initialisation calcule un jeu de paramètres ajustés qui n'est jamais utilisé ; le dictionnaire employé porte des valeurs qui ressemblent à des valeurs par défaut. Les deux diffèrent nettement, notamment sur le pas du réseau.	Lequel des deux jeux est le bon. À trancher en vérifiant la position du pic d'un laser connu.
Modes laser et hacheur	Le mode synchronisé abandonne proprement quand aucun trigger n'arrive, ce qui est vérifié, mais aucune acquisition n'a jamais été faite avec un laser réellement branché. Le mode hacheur est construit d'après la documentation du pilote.	Le câblage du hacheur sur le montage, à confirmer avant de s'y fier.
Tampon disque au-delà de 50 spectres	Une sauvegarde courte est validée de bout en bout, mais le vidage périodique du tampon vers le fichier temporaire ne l'est pas.	Un enregistrement continu de plusieurs centaines de spectres, pour franchir le seuil.
Relecture d'un fichier	L'écriture est vérifiée, la relecture par le chargement de fond ne l'est pas.	Boucler le cycle : sauvegarder, recharger, comparer.
Fichier de paramètres séparé	save_parameter () écrit un fichier distinct et n'est appelée de nulle part.	Est-elle encore voulue, maintenant que les paramètres accompagnent les spectres ?
Longueurs d'onde sans monochromateur	La Stresing renvoie alors le <i>nombre</i> de pixels au lieu d'un tableau d'indices, malgré ce qu'annonce son message.	Sans effet tant qu'un monochromateur est attaché, ce qui est le cas ici.

5. Vérifications effectuées

Chaque conclusion de ce rapport repose sur une mesure ou une trace, non sur une lecture du code seule. Les affirmations concernant le matériel ont été vérifiées sur l'instrument.

Vérification	Résultat
Journal d'événements Windows (WHEA)	12 403 erreurs PCIe corrigées en trois jours, disparues après un redémarrage complet. Piste écartée comme cause du blocage, mais à surveiller.
py - spy dump sur le processus figé	Pile d'appels arrêtée dans <code>DLLStartMeasurement_blocking()</code> : blocage confirmé, à l'intérieur du DLL du fabricant.
Acquisition directe, hors interface	Signal réel de 1010 points, valeurs de 0 à 142 comptes, variant entre deux acquisitions. La caméra fonctionnait ; le problème était en aval.
Axe des longueurs d'onde, monochromateur attaché	1010 points finis, de 720,9 à 769,0 nm, centrés sur les 747 nm rapportés par l'instrument. Longueur identique au spectre.
Chaîne d'affichage isolée	Reproduction de <code>IndexError</code> , puis vérification qu'après correction la courbe atteint le graphique avec des axes corrects.
Dialogue série brut avec le contrôleur	Preuve que 1 MIRROR répond ok sans agir, et que EXIT-MIRROR puis FRONT déplacent bien le miroir.
Construction complète de l'interface, hors écran	Fenêtre montée contre le matériel réel, onglets et connexions vérifiés.
Laser hélium-néon à 632,8 nm	Pic observé sur la Stresing, puis sur la Pixis après bascule du miroir, chacune devenant aveugle quand la lumière part vers l'autre port.
Soustraction de fond, sept scénarios	Correction ignorée sans fond ; fond mesuré puis appliqué ; fichier de plusieurs spectres accepté ; mauvaise longueur refusée ; fond abandonné au changement de spectromètre.
Libération du drapeau d'occupation	La progression atteint 100 % dans les trois cas : mesure normale, mesure qui lève une exception, mesure arrêtée avant de démarrer.
Fond à 0, 1 et 3 balayages	Les trois aboutissent, avec un axe de longueurs d'onde de taille correcte.
Enregistrement des paramètres	Valeurs consignées, clés absentes et lectures non numériques remplacées par la dernière valeur connue, matrice de paramètres enfin non nulle.
Chronométrage de la sauvegarde	21 ms au lieu des 500 ms systématiques, fichier écrit correctement.
Exports réels du DLL Stresing	Table d'exports lue dans le binaire : <code>DLLAbortMeasurement</code> existe, <code>DLLStopMeasurement</code> non. C'est ce qui a révélé la première régression.
Départ bloquant contre non bloquant	Mesuré sur la carte : le non bloquant renvoie 976, « An unknown error occurred », et rien ne démarre ; le bloquant rend la main en 0,11 s avec un spectre de 1024 points.
Contrôle du verrou Pixis	Trois threads démarrant et arrêtant l'acquisition pendant soixante changements de région : aucune erreur, là où ce schéma reproduisait <i>AcquisitionInProgress</i> .
Enregistrement complet	Acquisition puis sauvegarde, fichier rouvert : spectre de 1024 points avec son axe en longueur d'onde en attribut, 30 paramètres matériels dont le temps de pose, le réseau et le port de sortie, et les commentaires. Noms et lignes concordent.

Vérification	Résultat
Saturation du capteur Pixis	Balayage du temps de pose de 200 à 0,5 ms : la médiane de la trame chute de 65 535 à 1684 entre 200 et 50 ms, puis cesse de bouger. La pose atteint bien la caméra, relue depuis l'instrument.
Acquisition Stresing, après réparation	Mode continu : 0,17 s, 1010 points, jusqu'à 16383 comptes, deux fois de suite. Mode externe sans laser : abandon après le délai configuré, puis le mode continu fonctionne immédiatement, donc la carte n'est plus laissée occupée.

Réserves

Le mode synchronisé au laser et le mode hacheur n'ont **pas** été validés en conditions réelles : ils sont construits d'après la documentation du pilote, et la seule vérification matérielle disponible a été l'absence de trigger sur l'entrée I.

Les vérifications des correctifs de la chaîne d'acquisition ont été menées hors matériel, sur des spectromètres simulés. Elles prouvent la logique — qu'un fond absent n'est pas soustrait, qu'une mesure en échec libère l'interface — mais pas le comportement au contact des instruments réels. Une prise de données complète, du fond jusqu'au fichier sauvegardé, reste à faire.

Une observation qui appartient au montage, pas au code

Le balayage du temps de pose sur la Pixis a livré un résultat qui mérite d'être noté ici. Entre 200 et 50 ms, la médiane de la trame s'effondre comme attendu. En dessous de 10 ms, elle cesse de bouger : 1357, puis 1313, puis 1303 comptes, et environ 37 000 pixels restent à la saturation quelle que soit la pose. Une charge qui ne dépend pas de la durée d'exposition ne vient pas de l'exposition ; la lecture du capteur dure un temps fixe, et sur un capteur pleine trame la lumière continue de s'y accumuler. C'est la signature d'un maculage de lecture.

La conséquence pratique est qu'aucun réglage logiciel ne rattrapera une source trop intense : il faut atténuer optiquement. Un diaphragme ne suffit pas s'il coupe les bords d'un faisceau dont c'est le cœur qui sature. À noter également que le pilote déclare un attribut d'obturateur qu'il n'utilise jamais.

Ce que cette session suggère

Les défauts trouvés partagent un mode de défaillance unique : **l'échec silencieux**. Une exception envoyée sur la sortie d'erreur plutôt que dans le journal, une commande série à laquelle l'instrument répond ok sans rien faire, un appel bloquant sans échéance, un tableau non initialisé traité comme une donnée valide. Dans chaque cas le logiciel affichait un état parfaitement normal.

C'est ce qui rend ces bugs coûteux, bien plus que leur difficulté technique : aucun n'était compliqué à corriger une fois vu, et plusieurs étaient présents depuis longtemps. Trois habitudes les auraient rendus visibles beaucoup plus tôt — journaliser les exceptions des threads plutôt que les laisser partir sur la sortie d'erreur ; refuser d'agir sur une donnée dont la validité n'est pas établie, au lieu de continuer avec ce qui traîne en mémoire ; et vérifier le code de retour de chaque appel au DLL, y compris ceux qu'on croit anodins.

La démonstration la plus nette de ce dernier point est fournie par les deux régressions de la section 3.9, commises pendant cette session avec le reste de ce rapport déjà écrit. Un except `AttributeError` destiné à rendre le code tolérant a masqué un nom de fonction inexistant ; un code de retour non lu a masqué un démarrage qui échouait. La leçon n'est pas qu'il faut être plus attentif, mais que ce style d'appel — un DLL propriétaire dont les erreurs sont des valeurs de retour — demande une discipline explicite, sans quoi la panne devient invisible par construction.